

IITSAR2 313 – SYSTEM INTEGRATION AND ARCHITECTURE 2

LABORATORY REPORT: EXERCISE 2 — MICROSERVICES EDGE CASE TESTING

Members: Andura, Carla C.

Bermejo, Kate Nicole

Garcia, Sophia Christi

Labrador, Mariene E.

Roma, Sean Justin E.

Abstract

This laboratory exercise tested a Node.js/Express Business Logic API server. The test suite executed 17 sequential steps using Windows PowerShell and curl.exe to verify product retrieval, order placement, stock management, and system resets. The testing covered valid order processing, strict validation rule enforcement, and specific error code generation, including `VALIDATION_ERROR`, `INVALID_QUANTITY`, `PRODUCT_NOT_FOUND`, `OUT_OF_STOCK`, and `STOCK_EXCEEDED`. All steps confirmed the correct application of business rules, accurate HTTP status code mapping, and reliable stock persistence across multiple API calls.

I. INTRODUCTION

Robust API architecture requires strict business logic validation to ensure data integrity and prevent invalid transactions. While basic CRUD (Create, Read, Update, Delete) operations handle data storage, business logic APIs enforce real-world constraints—such as ensuring a customer cannot order more items than are currently in stock, or preventing orders with negative quantities.

This laboratory focuses on an inventory and ordering system that exposes four primary endpoints:

- `GET /api/products` for listing all products.
- `GET /api/products/:id` for retrieving a specific product.
- `POST /api/orders` for placing an order and deducting stock.
- `POST /api/reset` for resetting the database to a known baseline for testing.

The system must map specific business outcomes to the correct HTTP status codes:

- 201 Created: Returned for valid orders where stock is successfully updated.
- 200 OK: Returned for successful GET requests and database resets.
- 400 Bad Request: Returned for validation failures and business rule violations (e.g., insufficient stock).
- 404 Not Found: Returned when querying or ordering a product ID that does not exist.
- 500 Internal Server Error: Reserved for unexpected server crashes, which should not occur during controlled edge case testing.

II. METHODOLOGY/IMPLEMENTATION

The testing environment required two separate command-line interfaces. Terminal 1 was dedicated exclusively to running the Node.js server (npm run start) to ensure the process was not interrupted. Terminal 2 was used to execute the 17-step sequential test suite using the curl.exe command line tool. Using the explicit curl.exe executable on Windows avoids conflicts with PowerShell's native aliases.

Rule / Scenario	Test Step(s)	Status Code
Valid order	Step 3, 12	201 ✓
Stock updated (persisted)	Step 4, 13	200 ✓
Invalid product ID	Step 5	404 ✓
Quantity = 0	Step 6	400 ✓
Negative quantity	Step 7	400 ✓
Decimal / non-integer quantity	Step 8	400 ✓

Stock exceeded	Step 9	400 ✓
Missing productId	Step 10	400 ✓
Missing quantity	Step 11	400 ✓
Out of stock (after drain)	Step 15	400 ✓
Reset to baseline	Step 16	200 ✓

A. Reset and Pre-conditions

Before starting, previous text evidence files were cleared to ensure a clean testing environment. Upon startup, the backend automatically initializes a baseline inventory of five products (such as a Laptop, Smartphone, and Keyboard) with predefined stock levels. Furthermore, the system state can be reverted to this clean baseline at any time during execution using the dedicated `/api/reset` endpoint.

```
PS C:\Users\Sean\Desktop\SAR2> cd C:\Users\Sean\Desktop\SAR2\Lab3\server
PS C:\Users\Sean\Desktop\SAR2\Lab3\server> npm install

up to date, audited 70 packages in 2s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Service Startup

Run in this exact order:

1. Open Terminal 1 and run only the server command there.
2. Keep Terminal 1 running (do not run test commands in Terminal 1).
3. Open Terminal 2 and run all curl test steps there.
4. Use the exact absolute paths below (do not use relative cd .\Lab3\...).
5. If any step fails with connection error, re-run Terminal 1 startup first.

Terminal 1:

```
node.js v20.11.1
PS C:\Users\Sean\Desktop\SAR2\Lab3\server> cd C:\Users\Sean\Desktop\SAR2\Lab3\server
PS C:\Users\Sean\Desktop\SAR2\Lab3\server> npm run start

> lab3-business-logic-api@1.0.0 start
> node server.js

Lab 3 - Business Logic API Server
ITSAR2 313 - System Architecture & Integration 2

✓ Server running on: http://localhost:3000
✓ API Base URL: http://localhost:3000/api

Available Endpoints:
GET /api/products      - List all products
GET /api/products/:id  - Get product by ID
POST /api/orders       - Place an order
POST /api/reset        - Reset to baseline (testing)

Ready to accept curl requests...
```

Terminal 2 (run all curl commands here):

```
PS C:\Users\Sean\Desktop\SAR2> cd C:\Users\Sean\Desktop\SAR2\Lab3
PS C:\Users\Sean\Desktop\SAR2\Lab3> New-Item -ItemType Directory -Force docs\
evidence | Out-Null
PS C:\Users\Sean\Desktop\SAR2\Lab3> 
```

Quick smoke check from Terminal 2 (optional):

```
PS C:\Users\Sean\Desktop\SAR2\Lab3> curl.exe -i http://localhost:3000/
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 196
ETag: W/"c4-5NWe90ykVty5BH1+hb51uhcpdtE"
Date: Sun, 22 Mar 2026 13:01:32 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"Lab 3 - Business Logic API","status":"running","endpoints":{"p
roducts":"GET /api/products","product":"GET /api/products/:id","placeOrder
":"POST /api/orders","reset":"POST /api/reset"}}
PS C:\Users\Sean\Desktop\SAR2\Lab3>
```

Expected: HTTP 200 with status: "running".

If smoke check fails, do this recovery sequence and try again:

```
PS C:\Users\Sean\Desktop\SAR2\Lab3> Get-Process node -ErrorAction SilentlyContinue | Stop-Process -Force
PS C:\Users\Sean\Desktop\SAR2\Lab3> cd C:\Users\Sean\Desktop\SAR2\Lab3\server
PS C:\Users\Sean\Desktop\SAR2\Lab3\server> npm run start

> lab3-business-logic-api@1.0.0 start
> node server.js

Lab 3 - Business Logic API Server
ITSAR2 313 - System Architecture & Integration 2

✓ Server running on: http://localhost:3000
✓ API Base URL: http://localhost:3000/api

Available Endpoints:
GET /api/products          - List all products
GET /api/products/:id      - Get product by ID
POST /api/orders           - Place an order
POST /api/reset            - Reset to baseline (testing)

Ready to accept curl requests...
```

If port 3000 is already in use, stop old Node processes before restart:

```
Get-Process node -ErrorAction SilentlyContinue |
Stop-Process -Force
```

Test Suite Overview:

This test suite covers 12+ scenarios testing:

- ✓ Happy path (valid orders)
- ✓ All validation rules
- ✓ All error codes (VALIDATION_ERROR, INVALID_QUANTITY, PRODUCT_NOT_FOUND, OUT_OF_STOCK, STOCK_EXCEEDED)
- ✓ Stock persistence across orders
- ✓ Edge cases

Sequential Test Steps (Terminal 2)

Step 1 - GET all products (200)

```
PS C:\Users\Sean\Desktop\SAR2\Lab3> curl.exe -i http://localhost:3000/api/
products | Tee-Object .\docs\evidence\01-get-products.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent                                  Dload  Upload   Total   Spent    Left   Sp
eed
  0         0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--
100    246 100    246    0     0   8763      0 --:--:-- --:--:-- --:--:--
8785
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 246
ETag: W/"f6-zrCrYc0Td0czYfMl0popMpkbu5c"
Date: Sun, 22 Mar 2026 13:06:55 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[{"id":1,"name":"Laptop","price":999,"stock":10}, {"id":2,"name":"Smartphon
e","price":599,"stock":25}, {"id":3,"name":"Tablet","price":399,"stock":15}
, {"id":4,"name":"Monitor","price":299,"stock":8}, {"id":5,"name":"Keyboard"
,"price":79,"stock":50}]
```

Expected: HTTP 200, array of 5 products with initial stock levels.

Step 2 - GET specific product by ID (200)


```

PS C:\Users\Sean\Desktop\SAR2\Lab3> curl.exe -i http://localhost:3000/api/
products/1 | Tee-Object .\docs\evidence\02-get-product-1.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent                                  Dload  Upload   Total   Spent    Left   Sp
eed
   0     0    0     0     0     0      0      0  --:--:-- --:--:-- --:--:--
100   47  100   47     0     0   2738      0  --:--:-- --:--:-- --:--:--
2764
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 47
ETag: W/"2f-n9L80lqh0J/kXNHYGnK4AkpgJyw"
Date: Sun, 22 Mar 2026 13:08:24 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"id":1,"name":"Laptop","price":999,"stock":10}

```

Expected: HTTP 200, Laptop product with stock=10.

Step 3 - POST valid order (201)

```

PS C:\Users\Sean\Desktop\SAR2\Lab3> '{"productId":1,"quantity":2}' | curl.
exe -i -X POST "http://localhost:3000/api/orders" -H "Content-Type: applic
ation/json" --data-binary "@-" | Tee-Object .\docs\evidence\03-order-valid
.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent                                  Dload  Upload   Total   Spent    Left   Sp
eed
   0     0    0     0     0     0      0      0  --:--:-- --:--:-- --:--:--
100  148  100  118  100    30   2311    587  --:--:-- --:--:-- --:--:--
2960
HTTP/1.1 201 Created
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 118
ETag: W/"76-gl12rt90+vhz/L/vt62A6vejqr0"
Date: Sun, 22 Mar 2026 13:08:41 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"Order successful","product":"Laptop","quantity":2,"pricePerUni
t":999,"totalPrice":1998,"remainingStock":8}

```

Expected:

- HTTP 201
- Message: "Order successful"
- totalPrice: 1998 (999 * 2)

- remainingStock: 8 (10 - 2)

Step 4 - Verify stock was updated (200)

```
PS C:\Users\Sean\Desktop\SAR2\Lab3> curl.exe -i http://localhost:3000/api/
products/1 | Tee-Object .\docs\evidence\04-verify-stock-updated.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent                                  Dload  Upload   Total   Spent    Left   Sp
eed
   0      0    0     0     0     0     0     0 --:--:-- --:--:-- --:--:--
100    46 100    46     0     0  2853     0 --:--:-- --:--:-- --:--:--
2875
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 46
ETag: W/"2e-L8PtBm7S4Rb9bYqb2x6ClUic/t4"
Date: Sun, 22 Mar 2026 13:09:00 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"id":1,"name":"Laptop","price":999,"stock":8}
```

Expected: HTTP 200, Laptop stock=8 (stock persisted).

Step 5 - POST invalid product ID (404)

```
PS C:\Users\Sean\Desktop\SAR2\Lab3> '{"productId":9999,"quantity":1}' | cu
rl.exe -i -X POST "http://localhost:3000/api/orders" -H "Content-Type: app
lication/json" --data-binary "@-" | Tee-Object .\docs\evidence\05-order-in
valid-product-404.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent                                  Dload  Upload   Total   Spent    Left   Sp
eed
   0      0    0     0     0     0     0     0 --:--:-- --:--:-- --:--:--
100   110 100    77  100    33  4503   1930 --:--:-- --:--:-- --:--:--
6470
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 77
ETag: W/"4d-Hhw7rpZkyZ5a2eZe0tlv2R7lh0"
Date: Sun, 22 Mar 2026 13:09:25 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"PRODUCT_NOT_FOUND","message":"Product with ID 9999 does not exist"}
```

Expected:

- HTTP 404
- error: "PRODUCT_NOT_FOUND"
- Stock should remain unchanged

Step 6 - POST quantity = 0 (400)

```
PS C:\Users\Sean\Desktop\SAR2\Lab3> '{"productId":1,"quantity":0}' | curl.exe -i -X POST "http://localhost:3000/api/orders" -H "Content-Type: application/json" --data-binary "@-" | Tee-Object .\docs\evidence\06-order-qty-zero-400.txt
```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Cu
			Dload	Upload	Total	Spent	Left
0	0	0	0	0	0	--:--:--	--:--:--
100	124	100	94	100	30	5256	1677

```
7294
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 94
ETag: W/"5e-PcQwmpmOYIrnGdMGyPOMdv9/uj4"
Date: Sun, 22 Mar 2026 13:09:43 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"INVALID_QUANTITY","message":"Quantity must be a positive integer greater than zero"}
```

Expected:

- HTTP 400
- error: "INVALID_QUANTITY"
- message: "Quantity must be a positive integer greater than zero"

Step 7 - POST negative quantity (400)

```

PS C:\Users\Sean\Desktop\SAR2\Lab3> '{"productId":1,"quantity":-5}' | curl
.exe -i -X POST "http://localhost:3000/api/orders" -H "Content-Type: appli
cation/json" --data-binary "@-" | Tee-Object .\docs\evidence\07-order-qty-
negative-400.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent                                  Dload  Upload   Total   Spent    Left   Sp
eed
   0      0     0    0    0    0    0    0  --:--:-- --:--:-- --:--:--
100  125  100   94  100   31  5596   1845  --:--:-- --:--:-- --:--:--
7812
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 94
ETag: W/"5e-PcQwmpmpOYIrngdMGyPOMdv9/uj4"
Date: Sun, 22 Mar 2026 13:10:04 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"INVALID_QUANTITY","message":"Quantity must be a positive integer
greater than zero"}

```

Expected:

- HTTP 400
- error: "INVALID_QUANTITY"

Step 8 - POST non-integer quantity / decimal (400)

```

PS C:\Users\Sean\Desktop\SAR2\Lab3> '{"productId":1,"quantity":2.5}' | cur
l.exe -i -X POST "http://localhost:3000/api/orders" -H "Content-Type: appl
ication/json" --data-binary "@-" | Tee-Object .\docs\evidence\08-order-qty
-decimal-400.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent                                  Dload  Upload   Total   Spent    Left   Sp
eed
   0      0     0    0    0    0    0    0  --:--:-- --:--:-- --:--:--
100  100  100   68  100   32  3503   1648  --:--:-- --:--:-- --:--:--
5263
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 68
ETag: W/"44-kc873cJ0iPW0s9qyEtPnnaPL/FE"
Date: Sun, 22 Mar 2026 13:10:27 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"INVALID_QUANTITY","message":"Quantity must be an integer"}

```

Expected:

- HTTP 400
- error: "INVALID_QUANTITY"
- message: "Quantity must be an integer"

Step 9 - POST stock exceeded (400)

```
PS C:\Users\Sean\Desktop\SAR2\Lab3> '{"productId":1,"quantity":100}' | curl
1.exe -i -X POST "http://localhost:3000/api/orders" -H "Content-Type: appl
ication/json" --data-binary "@-" | Tee-Object .\docs\evidence\09-order-sto
ck-exceeded-400.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent
          Dload  Upload   Total   Spent    Left   Sp
eed
   0     0    0     0     0     0     0     0  --:--:-- --:--:-- --:--:--
100  119  100    87  100    32   2541    934  --:--:-- --:--:-- --:--:--
3500
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 87
ETag: W/"57-TlN6rLPcXG3X505TH3mXRqvySHU"
Date: Sun, 22 Mar 2026 13:10:48 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"STOCK_EXCEEDED","message":"Insufficient stock. Available: 8, Req
uested: 100"}
```

Expected:

- HTTP 400
- error: "STOCK_EXCEEDED"
- message: "Insufficient stock. Available: 8, Requested: 100"
- (Note: stock=8 from Step 3, not 10)

Step 10 - POST missing productId field (400)

```
PS C:\Users\Sean\Desktop\SAR2\Lab3> '{"quantity":5}' | curl.exe -i -X POST
"http://localhost:3000/api/orders" -H "Content-Type: application/json" --
data-binary "@-" | Tee-Object .\docs\evidence\10-order-missing-productId-4
00.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent
                                Dload  Upload  Total  Spent    Left  Sp
eed
  0     0    0     0     0     0     0     0  --:--:-- --:--:-- --:--:--
100  101  100   85  100   16  5269   991  --:--:-- --:--:-- --:--:--
6312
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 85
ETag: W/"55-4x00KAIhNQ7Ztr80JrNEs8se+fo"
Date: Sun, 22 Mar 2026 13:11:09 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"VALIDATION_ERROR","message":"Missing required fields: productId,
quantity"}
```

Expected:

- HTTP 400
- error: "VALIDATION_ERROR"
- message: "Missing required fields: productId, quantity"

Step 11 - POST missing quantity field (400)

```

PS C:\Users\Sean\Desktop\SAR2\Lab3> '{"productId":1}' | curl.exe -i -X POST
T "http://localhost:3000/api/orders" -H "Content-Type: application/json" -
- data-binary "@-" | Tee-Object .\docs\evidence\11-order-missing-quantity-4
00.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent                                  Dload  Upload   Total   Spent    Left  Sp
eed
   0     0     0     0     0     0     0     0  --:--:-- --:--:-- --:--:--
100  102  100    85  100    17   5069   1013  --:--:-- --:--:-- --:--:--
6375
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 85
ETag: W/"55-4x00KAIhNQ7Ztr80JrNEs8se+fo"
Date: Sun, 22 Mar 2026 13:11:25 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"VALIDATION_ERROR","message":"Missing required fields: productId,
quantity"}

```

Expected:
- HTTP 400
- error:

"VALIDATION_ERROR"

Step 12 - POST another valid order to test stock deduction (201)

```

PS C:\Users\Sean\Desktop\SAR2\Lab3> '{"productId":2,"quantity":5}' | curl.
exe -i -X POST "http://localhost:3000/api/orders" -H "Content-Type: applic
ation/json" --data-binary "@-" | Tee-Object .\docs\evidence\12-order-valid
-2.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent                                  Dload  Upload   Total   Spent    Left  Sp
eed
   0     0     0     0     0     0     0     0  --:--:-- --:--:-- --:--:--
100  153  100   123  100    30   7303   1781  --:--:-- --:--:-- --:--:--
9562
HTTP/1.1 201 Created
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 123
ETag: W/"7b-+2PRQQBp+rKpm5X4ImDFh5S1yj8"
Date: Sun, 22 Mar 2026 13:11:41 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"Order successful","product":"Smartphone","quantity":5,"pricePe
rUnit":599,"totalPrice":2995,"remainingStock":20}

```

Expected:

- HTTP 201
- Message: "Order successful"
- Product: "Smartphone"
- totalPrice: 2995 (599 * 5)
- remainingStock: 20 (25 - 5)

Step 13 - Verify Smartphone stock was updated (200)

```
PS C:\Users\Sean\Desktop\SAR2\Lab3> curl.exe -i http://localhost:3000/api/
products/2 | Tee-Object .\docs\evidence\13-verify-smartphone-stock.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent                                  Dload  Upload   Total   Spent    Left  Sp
eed
  0     0    0     0     0     0      0      0 --:--:-- --:--:-- --:--:--
100   51 100   51     0     0  3089     0 --:--:-- --:--:-- --:--:--
3187
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 51
ETag: W/"33-DD14H5BkHKoJtZxQxwn0eh84njU"
Date: Sun, 22 Mar 2026 13:12:02 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"id":2,"name":"Smartphone","price":599,"stock":20}
```

Expected: HTTP 200, Smartphone stock=20.

Step 14 - POST order for out-of-stock scenario (first drain Keyboard)

```
PS C:\Users\Sean\Desktop\SAR2\Lab3> '{"productId":5,"quantity":50}' | curl
.exe -i -X POST "http://localhost:3000/api/orders" -H "Content-Type: appli
cation/json" --data-binary "@-" | Tee-Object .\docs\evidence\14-order-drai
n-keyboard.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent
                        Dload  Upload   Total   Spent    Left  Sp
eed
   0      0    0     0     0     0     0     0  --:--:--  --:--:--  --:--:--
100  151 100   120  100    31   6489   1676  --:--:--  --:--:--  --:--:--
8388
HTTP/1.1 201 Created
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 120
ETag: W/"78-m1tssdgzhAX3BoD8/SN6j2ZdZPI"
Date: Sun, 22 Mar 2026 13:12:19 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"Order successful","product":"Keyboard","quantity":50,"pricePer
Unit":79,"totalPrice":3950,"remainingStock":0}
```

Expected:

- HTTP 201
- remainingStock: 0

Step 15 - Try to order from empty stock (400)

```
PS C:\Users\Sean\Desktop\SAR2\Lab3> '{"productId":5,"quantity":1}' | curl.exe -i -X POST "http://localhost:3000/api/orders" -H "Content-Type: application/json" --data-binary "@-" | Tee-Object .\docs\evidence\15-order-from-empty-stock-400.txt
```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload	Upload	Total	Spent	Left
0	0	0	0	0	0	--:--:--	--:--:--
100	101	100	71	100	30	4172	1763

```
6312
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 71
ETag: W/"47-Huq9fJ4SmEOVFX1vzjLlBaCRitY"
Date: Sun, 22 Mar 2026 13:12:43 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"OUT_OF_STOCK","message":"Product 'Keyboard' is out of stock"}
```

Expected:

- HTTP 400
- error: "OUT_OF_STOCK"
- message: "Product 'Keyboard' is out of stock"

Step 16 - Reset products to baseline (201)

```
PS C:\Users\Sean\Desktop\SAR2\Lab3> curl.exe -i -X POST "http://localhost:3000/api/reset" | Tee-Object .\docs\evidence\16-reset-products.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent                                  Dload  Upload   Total   Spent    Left   Sp
eed
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--
100  298  100  298    0     0 17674      0  --:--:-- --:--:-- --:--:-- 1
8625
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 298
ETag: W/"12a-8fGwi3ndgLuQylahUjvPs97YDqw"
Date: Sun, 22 Mar 2026 13:12:59 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"Products reset to baseline","products":[{"id":1,"name":"Laptop",
,"price":999,"stock":10},{id":2,"name":"Smartphone","price":599,"stock":
25},{id":3,"name":"Tablet","price":399,"stock":15},{id":4,"name":"Monito
r","price":299,"stock":8},{id":5,"name":"Keyboard","price":79,"stock":50}
]}
```

Expected:

- HTTP 200
- All products reset to original stock levels
- Laptop stock=10, Smartphone stock=25, etc.

Step 17 - Verify reset worked (200)

```
PS C:\Users\Sean\Desktop\SAR2\Lab3> curl.exe -i http://localhost:3000/api/
products | Tee-Object .\docs\evidence\17-verify-reset.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Cu
rrent                                  Dload  Upload   Total   Spent    Left   Sp
eed
  0         0     0      0      0      0     0      0  --:--:-- --:--:-- --:--:--
100    246    100    246     0      0 15594     0  --:--:-- --:--:-- --:--:-- 1
6400
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 246
ETag: W/"f6-zrCrYcOTd0czYfm10popMpkbu5c"
Date: Sun, 22 Mar 2026 13:13:28 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[{"id":1,"name":"Laptop","price":999,"stock":10}, {"id":2,"name":"Smartphon
e","price":599,"stock":25}, {"id":3,"name":"Tablet","price":399,"stock":15}
, {"id":4,"name":"Monitor","price":299,"stock":8}, {"id":5,"name":"Keyboard"
,"price":79,"stock":50}]
```

Expected: HTTP 200, all products back to initial stock.

Expected HTTP Status Codes:

- 201 Created** - Valid order, stock updated
- 200 OK** - GET requests, reset successful
- 400 Bad Request** - Validation/business rule violations
- 404 Not Found** - Product doesn't exist
- 500 Internal Server Error** - Server errors (should not occur)

III. EXPERIMENTAL FINDINGS / OBSERVATIONS

All 17 sequential test steps successfully verified the API's business logic constraints.

Happy Path and Persistence (Steps 1-4, 12-13): The initial queries confirmed the baseline stock, showing the Laptop with a stock of 10. When a valid order for 2 Laptops was placed in Step 3, the API correctly returned HTTP 201 with a calculated totalPrice of 1998 and remainingStock of 8. Step 4 confirmed that this stock deduction persisted in the database. A subsequent valid order for 5 Smartphones in Step 12 successfully deducted the stock from 25 down to 20.

Validation Errors (Steps 6-8, 10-11):

The API successfully intercepted malformed input data, returning HTTP 400 Bad Request for all formatting violations.

- Submitting a quantity of 0 (Step 6) or a negative quantity (Step 7) returned an `INVALID_QUANTITY` error, mandating a positive integer.
- Submitting a decimal value (Step 8) returned an `INVALID_QUANTITY` error, mandating an integer.
- Omitting required fields such as `productId` (Step 10) or `quantity` (Step 11) triggered a `VALIDATION_ERROR` detailing the missing payload requirements.

Business Logic Constraints (Steps 5, 9, 14-15): The system correctly enforced inventory reality. Attempting to order a non-existent product ID (Step 5) returned an HTTP 404 `PRODUCT_NOT_FOUND` error. Attempting to order 100 Laptops when only 8 remained (Step 9) triggered an HTTP 400 `STOCK_EXCEEDED` error, accurately calculating the deficit. After intentionally draining the Keyboard stock to 0 in Step 14, a subsequent order in Step 15 correctly returned an HTTP 400 `OUT_OF_STOCK` error.

System Reset (Steps 16-17): The `POST /api/reset` endpoint in Step 16 successfully cleared all simulated transactions and restored the baseline inventory data, confirming that automated test suites can reliably clean up their state.

IV. LAB DISCUSSIONS

The testing sequence highlights the difference between structural validation and logical validation in API design. Steps 10 and 11 enforce structural validation—ensuring the JSON payload contains the correct keys. Steps 6 through 9 enforce logical validation—ensuring the data makes mathematical and real-world sense (e.g., you cannot purchase a fraction of a physical product, nor can you purchase stock that does not exist).

By providing specific error string codes (like `STOCK_EXCEEDED` instead of a generic "Bad Request"), the API allows frontend client applications to handle exceptions programmatically. For example, a frontend could read the `STOCK_EXCEEDED` flag and automatically prompt the user to lower their cart quantity to match the available stock.

Additionally, the implementation of a dedicated `/api/reset` endpoint is a critical practice for testing environments. Because business logic APIs mutate state (deducting stock permanently), deterministic testing is impossible without a programmatic way to revert the data.

The following guide questions synthesize the core concepts demonstrated in this laboratory exercise. While the earlier sections focused on the technical execution of API routing and request handling, these questions explore the architectural importance of the Business Logic layer. Drawing directly from the edge cases tested—such as preventing negative inventory, handling out-of-stock scenarios, and strictly validating payload structures—the answers below outline why centralizing these rules in the backend is critical for overall system security, maintainability, and data integrity."

1. What is business logic in software architecture?

Business logic refers to the custom rules, algorithms, and constraints that dictate how data can be created, stored, and manipulated based on real-world business requirements. It acts as the bridge between the routing/controller layer and the database. In an e-commerce API, the business logic is the specific code that calculates the total order price (price * quantity), verifies if a requested product ID exists, and ensures the requested quantity does not exceed the remainingStock.

2. Why should business logic not be placed in the UI layer?

Placing business rules solely in the frontend (UI) creates severe security vulnerabilities because client-side code can be easily bypassed or manipulated by users. As demonstrated in this lab by using raw curl.exe commands, an API can be accessed directly, completely bypassing any UI checks. If the backend does not independently enforce the rules, malicious or malformed requests will execute successfully. Additionally, keeping logic in the backend ensures reusability; multiple client interfaces (e.g., a web app, a mobile app, and a third-party integration) can rely on the exact same centralized rules without developers having to rewrite the logic for each platform.

3. How does business logic improve data integrity?

Business logic acts as a strict gatekeeper before any data reaches the database. By validating input structure (e.g., ensuring a quantity is a positive integer rather than a decimal) and enforcing logical constraints (e.g., verifying an item is in stock before creating an order), it guarantees that the database only ever stores valid, consistent, and accurate information. It prevents mathematically impossible or logically flawed scenarios from being saved as permanent records.

4. What happens if business logic is not implemented?

Without implemented business logic, a system is defenseless against corrupted data, application crashes, and exploitation. In an inventory system, an absence of these rules would lead to catastrophic failures: stock levels dropping into negative numbers, users ordering non-existent items, checkout processes accepting \$0 or negative payments, and databases crashing due to missing required fields. Ultimately, the software would cease to reflect operational reality and become entirely untrustworthy.

V. CONCLUSIONS

The Business Logic API laboratory exercise successfully demonstrated the critical role of the backend in maintaining application data integrity. Through a rigorous 17-step testing sequence utilizing curl.exe, the Express.js microservice proved its resilience against a variety of structural and logical edge cases.

The API consistently returned appropriate HTTP status codes to reflect exact business outcomes: successfully persisting valid transactions (201 Created), gracefully rejecting malformed structural payloads like missing fields (400 Bad Request / VALIDATION_ERROR), denying mathematically impossible orders like negative quantities (400 Bad Request / INVALID_QUANTITY), and strictly

enforcing real-world inventory limits (400 Bad Request / STOCK_EXCEEDED and OUT_OF_STOCK). Furthermore, the inclusion of a dedicated /api/reset endpoint proved essential for establishing a deterministic testing environment, allowing the database to be predictably reverted to its baseline state.

Ultimately, this exercise highlighted that front-end UI validation is merely a usability feature, whereas backend business logic is a fundamental security requirement. By centralizing these constraints within the API layer, the system ensures that no client—whether a legitimate web frontend or a raw command-line tool—can force the database into an invalid state.